Functional Programming

Revision Lab 01

V. Markos

Mediterranean College

First Functional Programming revision lab, with a focus on recursion, sorting, and…pancakes. The main aim of this lab is to help reorganise any concepts explored so far.

Warm Up

A few exercises to get us started, with a focus on recursion and simple list operations. As a set of guidelines for this and all upcoming exercises, do use function signatures and, where needed, utilise list comprehensions appropriately.

Exercise 1. Write a function named `max'` that returns the maximum element of a list. Then implement a tail recursive version, `max''`.

Exercise 2. Write a function named `sum'` that computes the sum of the elements of a list of floats. Then implement a tail recursive version, `sum''`.

Exercise 3. Write a function named `fib` that computes the n–th term of the Fibonacci sequence. As a reminder the n–th term of the Fibonacci sequence is given by:

$$f_n = f_{n-1} + f_{n-2}, \tag{1}$$

with $f_0 = 0$ and $f_1 = 1$. The first ten terms, thus, read:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34.$$

As you might have observed, running the function `fib` for large inputs is quite slow. Can you explain why?

Exercise 4. As `fib` is too slow, given duplicated recursive function calls, we should devise a more optimal solution. To do so, implement a *tail recursive* solution, where you use auxiliary arguments appropriately to keep track of the current Fibonacci value and anything else you might find useful. You can use `:set +s` in GHCi to print timing and memory data. What do you observe?

Merge Sort

Having recalled recursion basics, we now proceed to have a bit of higher order fun with a classical sorting algorithm you have encountered last year in your Data Structures module. With that in mind, the next few exercises will help us implement this algorithm from scratch. A simple but key operation during merge sort is list splitting at two (almost) equal halves.

Exercise 5. Write a function named `splitAt'` that given a non–negative integer `n` and a list `ls` splits `ls` at two parts at `n`, returning a 2–tuple containing the two parts.

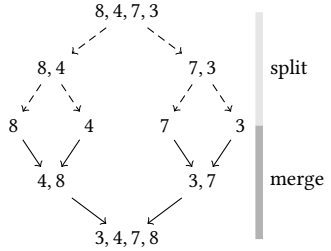

Figure 1: Merge sort algorithm. First the list is recursively split into halves which are then merged in a way that respects order of each merged part.

Another key operation needed to implement merge sort is order respecting list merging, i.e, merging two *sorted* lists providing a new *sorted* list as an output.

Exercise 6. Implement a sorted list merging algorithm as a Haskell function `merge`, that accepts two sorted lists and outputs a merged sorted list, as described above. As an example, for `a=[2,4,7]` and `b=[1,5,6]` the output should be `merge a b == [1,2,4,5,6,7]`. You might use Figure 2 as a rough guide for your implementation.

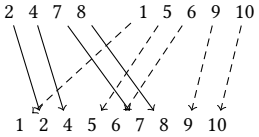

Figure 2: Sorted list merging. A typical imperative implementation would invoke a loop over both lists, using appropriate indices for each list. What about recursion?

Having implemented the key operation of sorted list merging and list splitting, the actual implementation of merge sort requires me(a)gre effort[1]. So, since we are at it:

Exercise 7. Implement merge sort as a Haskell function `mergeSort`. You can (and, maybe, should), utilise `merge` from Exercise 6 and `splitAt'` from Exercise 5. Consider how splitting and then merging will be implemented using recursion.

[1] Cheap wordplay with merge and meagre, in case it went unnoticed…

Pancake Sorting

Do you like pancakes? Well, regardless of your likings, you are requested to sort a heap of pancakes, as the one shown in Figure 3. More formally, assume you are given a heap of n pancakes, with radiuses from 1 up to n (pairwise different) placed one on top of the other at any order. Left here, the problem is efficiently (optimally, actually) addressed by merge sort. However, since we are talking about pancakes, we further assume that the only way we are allowed to reorder them is by using a spatula. Thus, we can only flip any top part of the pancake heap.

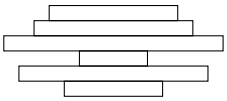

Figure 3: A heap of pancakes, awaiting to be sorted.

Exercise 8. Devise an algorithm that solves the pancake sorting problem described above, i.e., sorts a heap of pancakes by just flipping its top parts (of any length). You can also try to describe it in pseudocode or even implement it in any language you feel more comfortable with, besides Haskell, of course, to verify that your approach actually works.

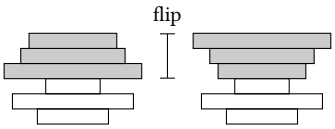

Figure 4: A valid "spatula flip" for pancake sorting.

Exercise 9. Implement the algorithm you developed in Exercise 8 as a Haskell function using recursion. Before proceeding to the actual implementation, try to figure out the following:

- What should be the base case for this problem?
- How can the solution for n be reduced to the solution for some smaller value of n, e.g., $n-1$?

You can hard–code some pancake permutations in order to test your implementation.

Random Permutations vs Sorting

So far, we have generated a random permutation by hard–coding it into our codebase. However, a lot of interesting and efficient algorithms do

exist for generating random permutations of lists of integers. Probably the most well known, Fisher-Yates, is a common efficient way to implement random shuffling in $O(n)$. However, its implementation requires the active utilisation of static arrays, which are not that straightforward to implement in Haskell. As a substitute, we can think of random shuffling as essentially sorting towards a not necessarily linearly sorted list (in the ordinary sense). Thus, we can utilise any sorting algorithm, in particular merge sort, as developed in Exercise 7, to implement an almost equally efficient random shuffling algorithm in Haskell, with the added advantage of guaranteed lack of impurities — besides randomness, of course.

Exercise 10. Think of an algorithm that could be utilised in order to shuffle a list by improvising on merge sort. The key here is to ensure equal probability across all possible permutations. *Hint:* Just choosing at random when merging does not suffice. Can you see why?

Exercise 11. Implement the algorithm you devised in Exercise 10. To do so, you will need a way to generate random numbers in Haskell, which can be efficiently done using the `Random` monad[2] from the `Control` class, i.e., `Control.Monad.Random`. To generate random integers in a closed range $[a, b]$, just use `getRandomR (a, b)`. You might also want to consider wrapping everything around a custom monad, by declaring the type `type R a = Rand StdGen a` or simply using the `IO` monad.

[2] A monad is something we are going to study later on this semester. Till then, you can think of them as "wrappers" that ensure some certain sort of impurity (e.g., user input / output) stays within certain boundaries, not "polluting" the rest of the program.